



用20行代码从头写强化学习

张晗

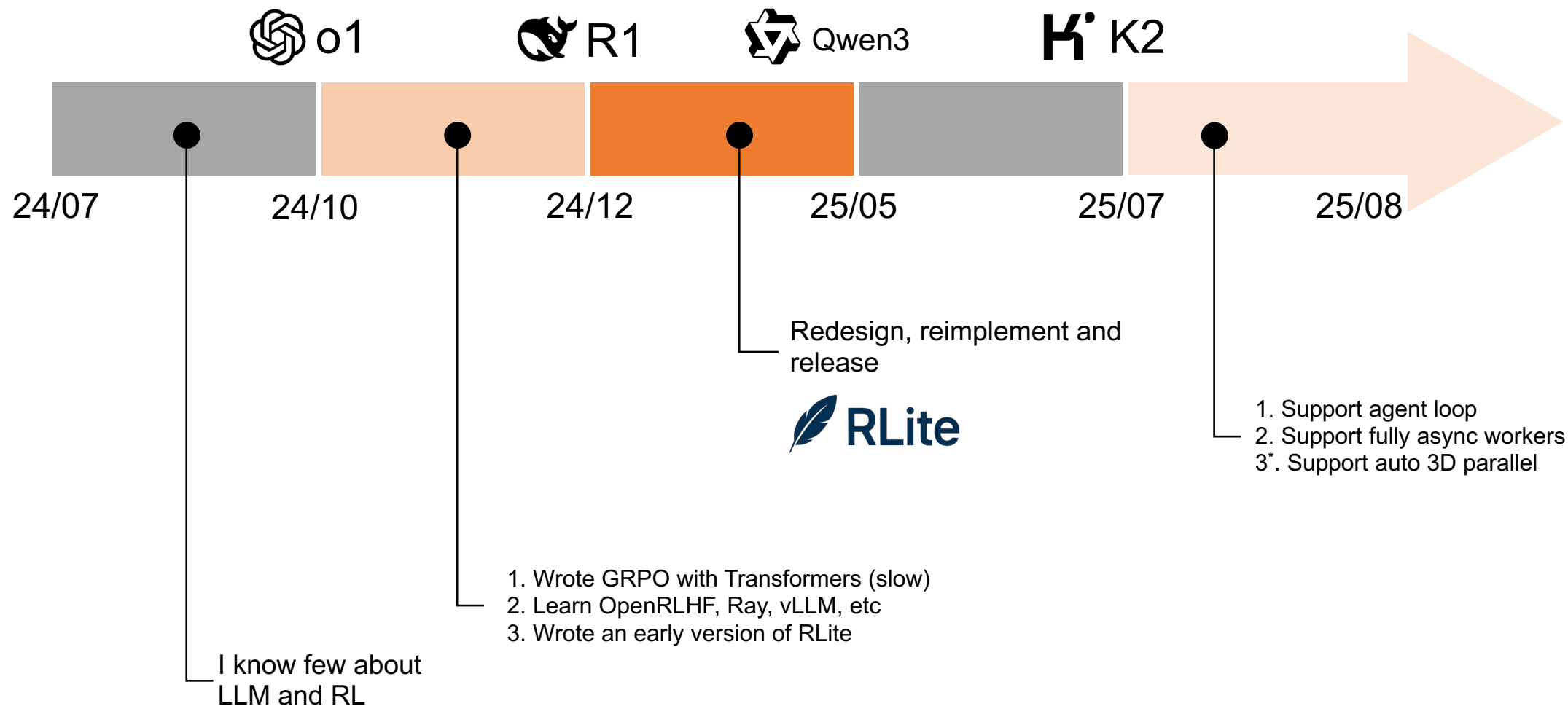
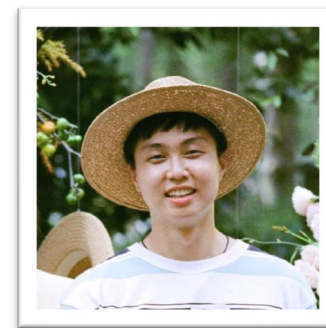
<https://github.com/r-lite-project/RLite>

<https://github.com/r-lite-project/RLite-Recipe>

Background

OPENRLHF

VeRL



Write GRPO with HF Transformers



```
5  def train_step():
6      # 1. Model and optimizer
7      model = transformers.AutoModelForCausalLM.from_pretrained("Qwen/Qwen2.5-7B")
8      optimizer = optim.Adam(model.parameters(), lr=1e-5)
9
10     # 2. Data
11     prompts, answers = prepare_data()
12     options = {"n": 8}
13
14     # 3. Generation and advantage
15     generations = model.generate(prompts, **options) Slow!
16     grpo_scores = calculate_varifiable_rewards(prompts, answers)
17
18     # 4. Train
19     loss = calculate_gradient_policy(generations, grpo_scores)
20     loss.backward()
21     optimizer.step()
22     optimizer.zero_grad()
```

Write GRPO with RLite



1. Define an nn.Module

```
7 class RliteQwen2TrainModule(rlite.nn.HuggingFaceFsdp2TrainModule, transformers.Qwen2ForCausalLM):
8     def on_weights_materialized(self):
9         self.optimizer = optim.Adam(self.parameters(), lr=1e-5)
10
11     def calculate_gradient_policy_and_update(self, generations, grpo_scores):
12         loss = calculate_gradient_policy(generations, grpo_scores)
13         loss.backward()
14         self.optimizer.step()
15         self.optimizer.zero_grad()
```

Write GRPO with RLite



2. An updated train step

```
18 def train_step():
19     # 1. Data
20     prompts, answers = prepare_data()
21     options = {"n": 8}
22
23     # 2. Rollout model
24     rollout_model = rlite.RliteInferenceEngine("Qwen/Qwen2.5-7B", executor="vllm") # On GPU now
25
26     # 3. Generation and advantage
27     generations = rollout_model.generate(prompts, **options)
28     grpo_scores = calculate_varifiable_rewards(prompts, answers)
29
30     # 4. Train
31     ## 4.1 Release GPU memory of rollout model
32     rollout_model.meta()
33     ## 4.2 Create actor model
34     with accelerate.init_empty_weights():
35         meta_module = RliteQwen2TrainModule.from_pretrained(
36             "Qwen/Qwen2.5-7B", torch_dtype="bfloat16"
37         )
38     actor_model = rlite.RliteTrainEngine(meta_module, executor="fsdp2") # Similar to FSDP wrapper
39     ## 4.3 Do training
40     actor_model.calculate_gradient_policy_and_update(generations, grpo_scores)
41
42     # 5. Sync weights
43     sync_weights(actor_model, rollout_model)
```

Write GRPO with RLite



3. Sync weights

```
46 def sync_weights(actor_model, rollout_model):
47     rollout_model.cuda("weights") # Load weights to GPU only (kind of misleading, will be fixed)
48     actor_model.p2p_weight_sync(rollout_model) # GPU-to-GPU weight sync via CUDAIPC
49     actor_model.cpu()
50     rollout_model.cuda("kv_cache") # The KV cache of vLLM is back to GPU
```

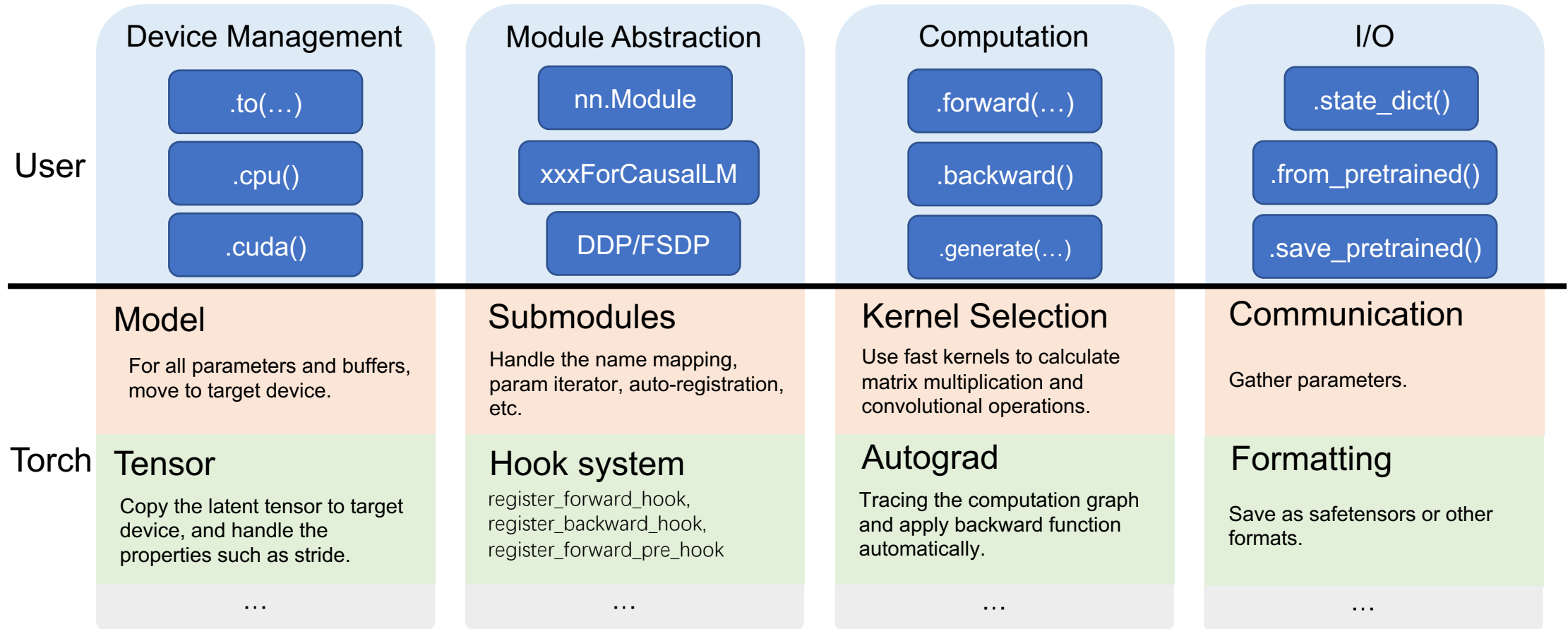
Write GRPO with RLite



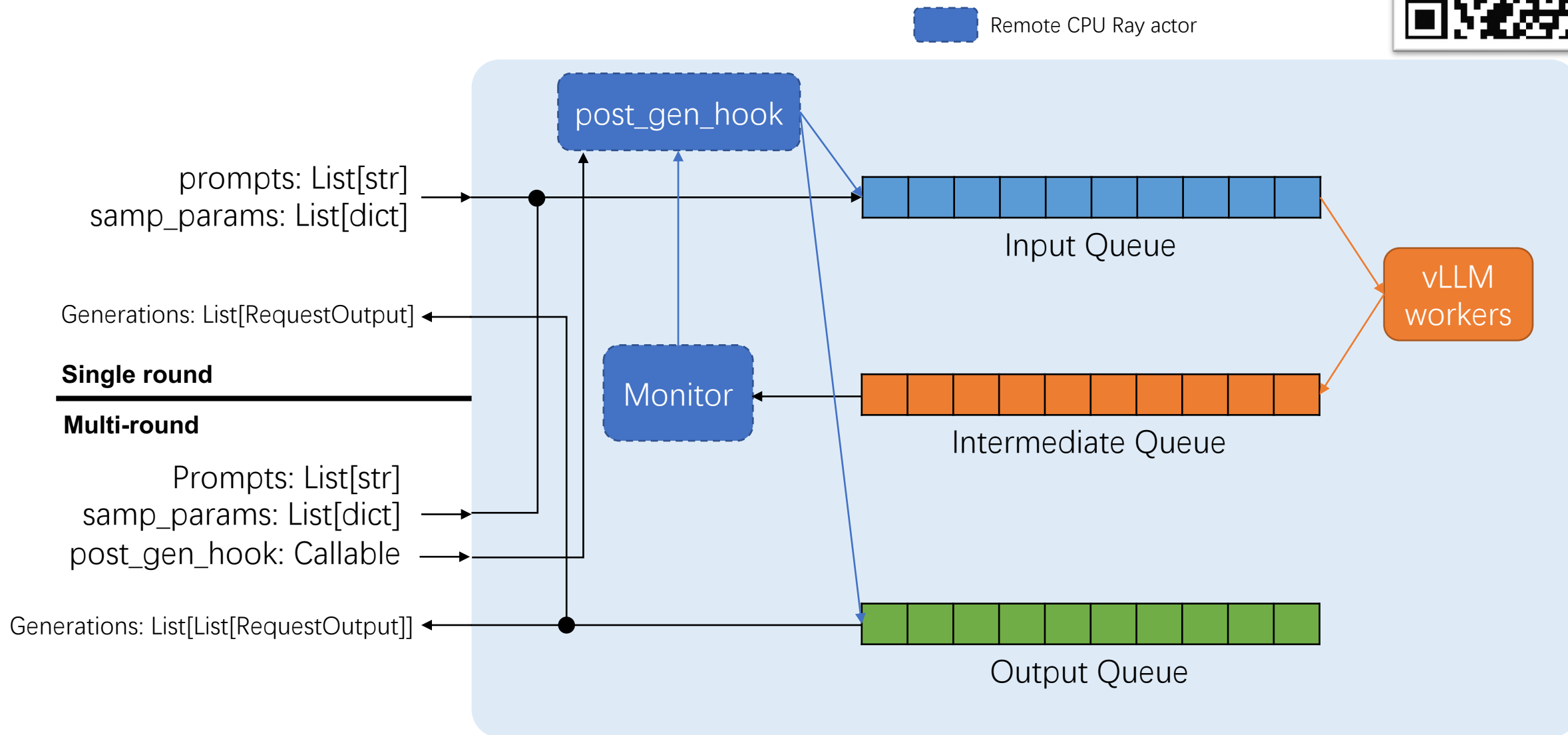
```
5 def train_step():
6     # 1. Data
7     prompts, answers = prepare_data()
8     options = {"n": 8}
9
10    # 2. Model and optimizer
11    model = transformers.AutoModelForCausalLM.from_pretrained("Qwen/Qwen2.5-7B")
12    optimizer = optim.Adam(model.parameters(), lr=1e-5)
13
14    # 3. Generation and advantage
15    generations = model.generate(prompts, **options)
16    grpo_scores = calculate_varifiable_rewards(prompts, answers)
17
18    # 4. Train
19
20    # 5. Sync weights
21    sync_weights(actor_model, rollout_model)
22
```

```
18 def train_step():
19     # 1. Data
20     prompts, answers = prepare_data()
21     options = {"n": 8}
22
23    # 2. Rollout model
24    rollout_model = rlite.RliteInferenceEngine("Qwen/Qwen2.5-7B", executor="vllm") # On GPU
25
26    # 3. Generation and advantage
27    generations = rollout_model.generate(prompts, **options)
28    grpo_scores = calculate_varifiable_rewards(prompts, answers)
29
30    # 4. Train
31    ## 4.1 Release GPU memory of rollout model
32    rollout_model.meta()
33    ## 4.2 Create actor model
34    with accelerate.init_empty_weights():
35        meta_module = RliteQwen2TrainModule.from_pretrained(
36            "Qwen/Qwen2.5-7B", torch_dtype="bfloat16"
37        )
38    actor_model = rlite.RliteTrainEngine(meta_module, executor="fsdp2") # Similar to FSDP w
39    ## 4.3 Do training
40    actor_model.calculate_gradient_policy_and_update(generations, grpo_scores)
41
42    # 5. Sync weights
43    sync_weights(actor_model, rollout_model)
44
```

What I want to say: Interface Matters!



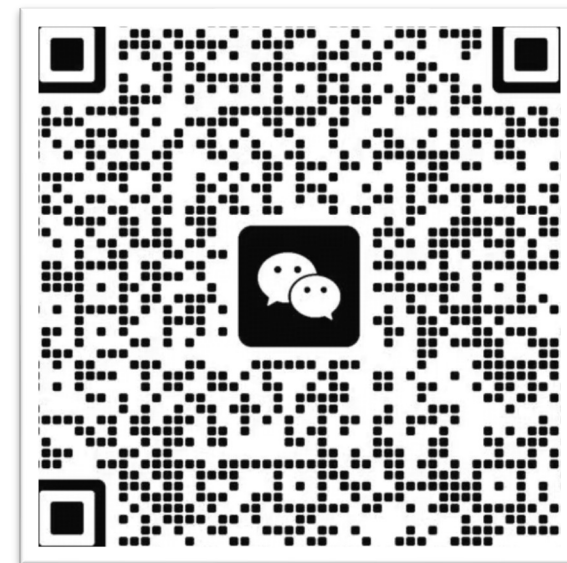
Multi-round Rollout



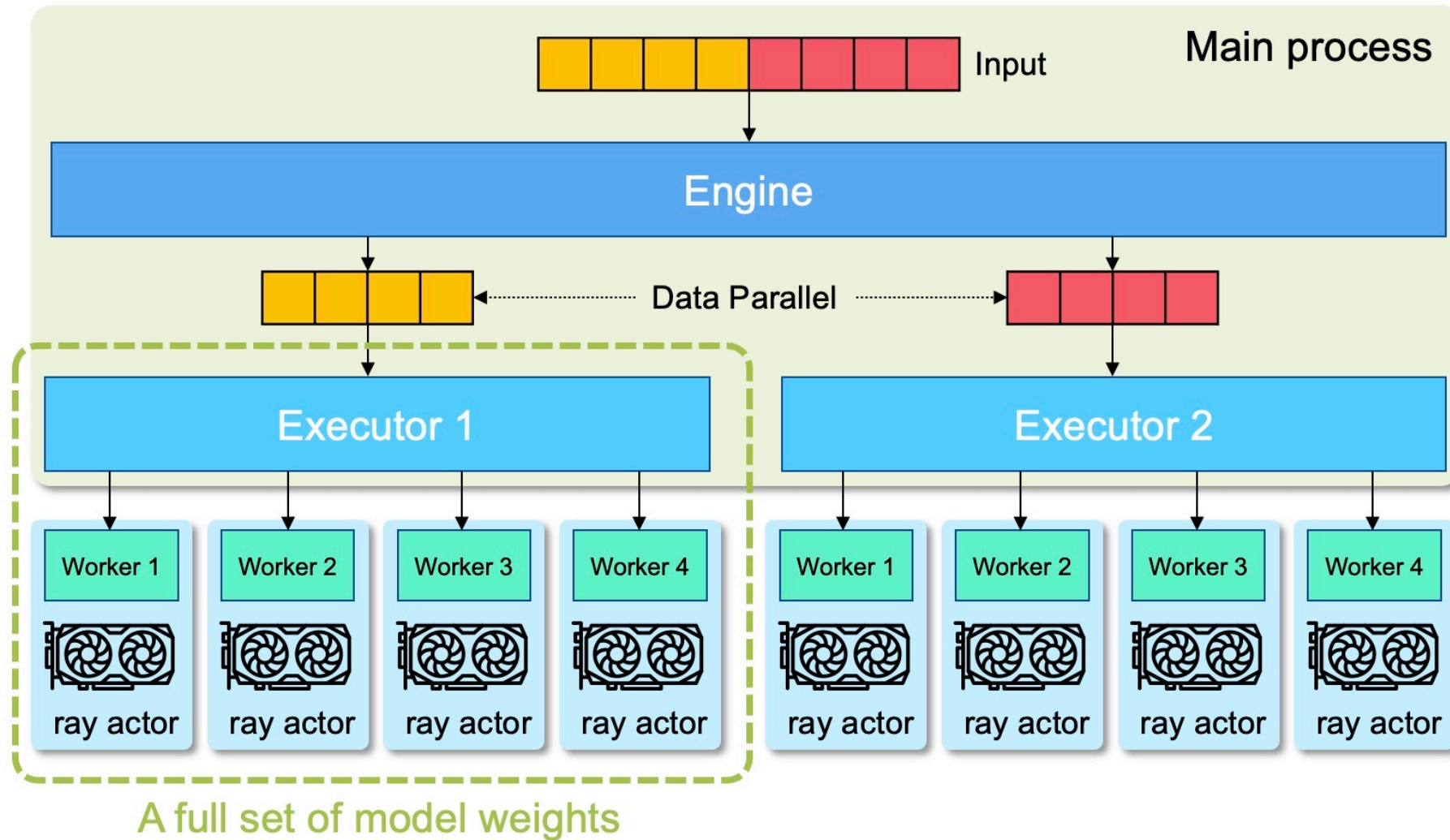
* Currently, the generations for the single round is a list of length 1.

Thanks! Q&A

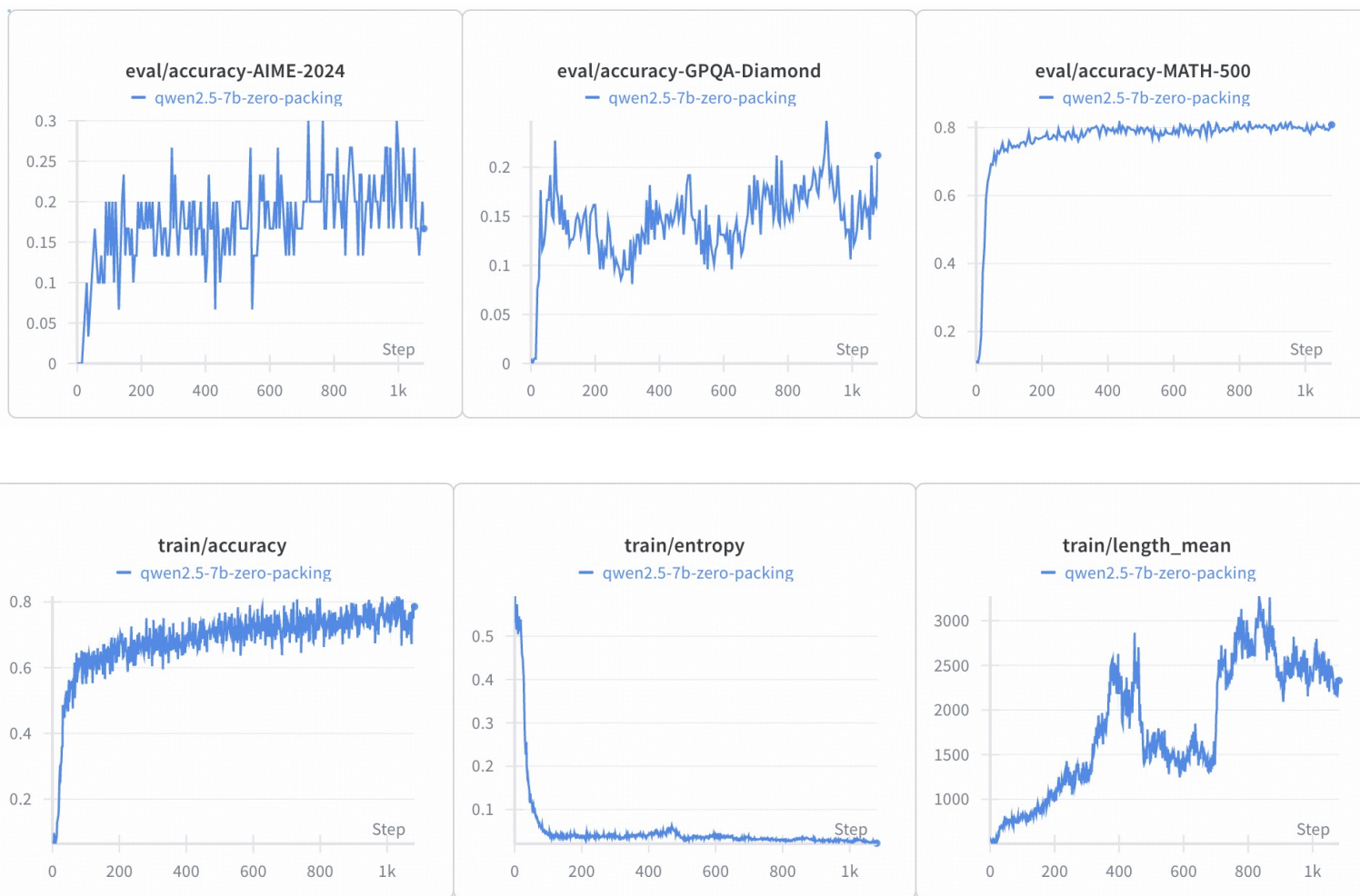
Welcome to join our discussion on WeChat!



Behind the Scene



A 500-line Repro. of ORZ (Zero RLVR)



- Including all logs
- A single file
- Qwen2.5-7B